La Realizzazione del Primo Chatbot in Python

Python per l'intelligenza artificiale generativa: costruire un chatbot funzionante

1. Che cos'è un chatbot e perché costruirne uno proprio

1.1 Definizione di chatbot e sua evoluzione con l'IA generativa

Un chatbot è, in termini generali, un programma capace di sostenere una conversazione testuale con una persona, rispondendo a messaggi ricevuti in un formato il più possibile naturale: i primi chatbot, diffusi già da diversi decenni, si basavano generalmente su regole predefinite e risposte precostituite, capaci di gestire soltanto un numero limitato di richieste previste in anticipo da chi li aveva sviluppati, con risultati spesso rigidi e facilmente riconoscibili come non genuinamente conversazionali da parte di chi vi interagiva.

L'avvento dei moderni modelli linguistici, già descritti nei moduli precedenti di questo percorso, ha profondamente trasformato questo scenario: un chatbot costruito attorno a un modello linguistico non si limita a selezionare tra risposte precostituite, ma genera dinamicamente una risposta pertinente a partire dal messaggio ricevuto, permettendo conversazioni molto più naturali, flessibili e capaci di adattarsi a richieste non previste esplicitamente in fase di sviluppo.

1.2 Perché costruire un proprio chatbot invece di usare uno strumento già pronto

Chi gestisce una piccola attività potrebbe chiedersi, legittimamente, perché investire tempo nella realizzazione di un proprio chatbot, quando esistono già numerosi strumenti di intelligenza artificiale

generativa pronti all'uso, accessibili tramite semplice interfaccia web: la risposta risiede nella possibilità di personalizzazione che un chatbot sviluppato autonomamente permette, sia in termini di comportamento e conoscenze specifiche del proprio settore, sia in termini di integrazione diretta all'interno dei propri strumenti di lavoro già esistenti, come il proprio sito internet o un canale di comunicazione già utilizzato con i propri clienti.

Un chatbot sviluppato su misura può inoltre essere configurato per rispondere esclusivamente secondo determinati vincoli, mantenere un tono coerente con l'immagine della propria attività, e limitare le proprie risposte agli argomenti di effettivo interesse, un livello di controllo generalmente non disponibile quando ci si limita a utilizzare uno strumento generico già pronto messo a disposizione da terzi.

1.3 Che cosa serve per iniziare questo progetto pratico

Per realizzare il primo chatbot descritto in questo modulo è necessario disporre degli stessi elementi già introdotti nei moduli precedenti di questo percorso: un'installazione funzionante di Python sul proprio computer, il gestore di pacchetti pip per installare le librerie necessarie, e una chiave di accesso valida per l'API del fornitore di modelli linguistici prescelto, custodita secondo le buone pratiche di sicurezza già illustrate nel modulo dedicato all'utilizzo delle API.

Non è necessario disporre di competenze avanzate di programmazione per seguire questo percorso pratico: il codice presentato nelle prossime sezioni è volutamente mantenuto semplice ed essenziale, pensato per essere comprensibile anche da chi possiede una conoscenza soltanto di base del linguaggio Python, privilegiando la chiarezza dei singoli passaggi rispetto alla completezza di

un'applicazione professionale già pronta per un utilizzo in produzione su larga scala.

In pratica

- Un chatbot basato su un modello linguistico genera risposte dinamiche, non precostituite
- Un chatbot personalizzato permette un controllo su tono, contenuti e integrazione
- Verificare di avere Python, pip e una chiave di accesso valida prima di iniziare
- Seguire il percorso pratico anche con una conoscenza soltanto di base di Python

2. Progettare il proprio chatbot prima di scrivere il codice

2.1 Definire lo scopo del chatbot

Prima di iniziare a scrivere una sola riga di codice, è buona pratica definire con chiarezza lo scopo specifico che il proprio chatbot dovrà svolgere: rispondere alle domande più frequenti dei propri clienti, guidare una persona nella scelta di un prodotto, raccogliere alcune informazioni preliminari prima di un contatto diretto, sono tutti esempi di scopi ben definiti che orientano concretamente le scelte successive relative al comportamento e alle istruzioni fornite al modello linguistico sottostante.

Un chatbot con uno scopo definito in modo vago o eccessivamente ampio tende a produrre risultati meno soddisfacenti rispetto a un chatbot pensato per un compito circoscritto e ben identificato: per questa ragione, è generalmente consigliabile, specialmente per un

primo progetto, concentrarsi su un obiettivo specifico e ben delimitato, ampliando eventualmente le funzionalità soltanto in un momento successivo, una volta verificato il buon funzionamento della versione iniziale.

2.2 Scegliere il modello e il fornitore più adatti

Come già descritto nel modulo dedicato all'utilizzo delle API, la scelta del fornitore e della specifica versione del modello da utilizzare dipende da diversi fattori, tra cui la qualità delle risposte richiesta dal proprio caso d'uso, il costo previsto in base al volume di conversazioni atteso, e la familiarità già acquisita con un determinato SDK ufficiale: per un primo progetto dimostrativo come quello descritto in questo modulo, è generalmente sufficiente scegliere un modello di fascia economica messo a disposizione dal fornitore già utilizzato negli esempi dei moduli precedenti, riservando l'eventuale passaggio a un modello più capace a un momento successivo, qualora le esigenze del progetto lo richiedano concretamente.

2.3 Definire il comportamento tramite le istruzioni di sistema

Come illustrato nel modulo precedente, il ruolo di sistema all'interno della struttura a messaggi di una richiesta permette di fornire al modello istruzioni generali valide per l'intera conversazione: nella progettazione del proprio chatbot, questa istruzione di sistema rappresenta lo strumento principale attraverso cui definire concretamente lo scopo, il tono e i vincoli di comportamento individuati nella fase di progettazione appena descritta, ad esempio specificando che il chatbot deve rispondere esclusivamente a domande relative ai prodotti e servizi della propria attività,

mantenendo un tono cordiale e professionale coerente con la propria immagine.

Scrivere un'istruzione di sistema efficace richiede generalmente alcuni tentativi successivi: è buona pratica iniziare con un'istruzione semplice e chiara, osservare concretamente il comportamento del chatbot su alcune conversazioni di prova rappresentative dell'utilizzo previsto, e affinare progressivamente l'istruzione stessa sulla base dei risultati osservati, un processo iterativo che porta generalmente, in tempi relativamente contenuti, a un comportamento soddisfacente per lo scopo prefissato.

In pratica

- Definire con chiarezza lo scopo specifico del proprio chatbot prima di scrivere codice
- Scegliere un modello di fascia economica per un primo progetto dimostrativo
- Usare l'istruzione di sistema per definire scopo, tono e vincoli di comportamento
- Affinare l'istruzione di sistema in modo iterativo osservando conversazioni di prova

3. La struttura di base del codice di un chatbot

3.1 Impostare l'ambiente di lavoro

Il primo passaggio pratico consiste nell'installare, tramite il gestore di pacchetti pip già descritto nel primo modulo di questo percorso, l'SDK ufficiale del fornitore prescelto, ad esempio la libreria openai, e nel configurare correttamente la propria chiave di accesso come variabile

d'ambiente, seguendo le buone pratiche di sicurezza già illustrate nel modulo precedente: completati questi passaggi preliminari, il proprio ambiente di lavoro è pronto per iniziare lo sviluppo vero e proprio del chatbot.

È generalmente consigliabile, per chi si avvicina per la prima volta a questo tipo di sviluppo, creare un nuovo file dedicato esclusivamente a questo progetto, mantenendo separato il codice del chatbot da eventuali altri esperimenti già realizzati nel corso dei moduli precedenti, in modo da poter procedere con ordine e individuare più facilmente eventuali errori nel corso dello sviluppo.

3.2 Inviare il primo messaggio e ricevere una risposta

Il nucleo minimo di un chatbot consiste in una funzione capace di inviare un messaggio al modello linguistico prescelto e restituire il testo della risposta ottenuta: questa operazione, già illustrata nel modulo precedente dedicato all'utilizzo delle API, rappresenta il mattone fondamentale su cui costruire progressivamente le funzionalità aggiuntive descritte nelle prossime sezioni di questo modulo, come la gestione della cronologia della conversazione e un'interfaccia utilizzabile da una persona.

Di seguito è riportato un primo esempio essenziale, che invia un singolo messaggio al modello e ne stampa a video la risposta ottenuta, utilizzando la libreria openai già introdotta nei moduli precedenti di questo percorso a titolo di esempio, fermo restando che la stessa logica si applica, con minime variazioni di sintassi, anche utilizzando l'SDK di un fornitore diverso.

```
from openai import OpenAI
```

```
client = OpenAI()
istruzione_sistema = "Sei l'assistente virtuale
di una libreria"

risposta = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content":
istruzione_sistema},
        {"role": "user", "content": "Avete libri
di cucina?"}
    ]
)

print(risposta.choices[0].message.content)
```

3.3 Organizzare il codice in una funzione riutilizzabile

Una volta verificato il corretto funzionamento dell'esempio essenziale appena illustrato, è buona pratica di programmazione racchiudere questa logica all'interno di una funzione dedicata, che riceva come parametro il messaggio da inviare e restituisca come risultato la risposta ottenuta: questa organizzazione del codice, oltre a renderlo più leggibile e ordinato, ne semplifica notevolmente il riutilizzo nelle sezioni successive di questo modulo, in cui questa stessa funzione verrà richiamata ripetutamente all'interno di una conversazione più articolata.

In pratica

- Installare l'SDK ufficiale e configurare la chiave di accesso come variabile d'ambiente
- Creare un file dedicato per il proprio progetto di chatbot

- Verificare il funzionamento dell'invio di un singolo messaggio prima di procedere
- Racchiudere la logica di invio in una funzione riutilizzabile e ordinata

4. Gestire una conversazione con memoria

4.1 Il problema della memoria tra le richieste

Come già anticipato nel modulo dedicato all'utilizzo delle API, un modello linguistico non conserva alcuna memoria autonoma tra una richiesta e la successiva: un chatbot che si limitasse a inviare, a ogni nuovo messaggio ricevuto da chi lo utilizza, esclusivamente quell'ultimo messaggio, senza alcun riferimento a quanto già scambiato in precedenza, apparirebbe a chi vi interagisce come privo di qualunque continuità conversazionale, incapace di fare riferimento a informazioni già fornite pochi istanti prima.

Per costruire un chatbot capace di sostenere una conversazione coerente nel tempo, è quindi necessario che il programma stesso si occupi di conservare e trasmettere, a ogni nuova richiesta, l'intera cronologia rilevante della conversazione già avvenuta, un compito che ricade interamente su chi sviluppa l'applicazione, dato che, come già chiarito, questa responsabilità non è delegabile al modello linguistico stesso.

4.2 Costruire e aggiornare la cronologia dei messaggi

In termini pratici, la cronologia di una conversazione può essere rappresentata, in Python, come una lista che raccoglie progressivamente tutti i messaggi scambiati, ciascuno con il proprio ruolo già descritto nel modulo precedente: a ogni nuovo messaggio

inviato dalla persona che utilizza il chatbot, questo viene aggiunto alla lista con il ruolo utente, la lista completa viene trasmessa al modello, e la risposta ottenuta viene a sua volta aggiunta alla stessa lista con il ruolo assistente, preparando così la lista aggiornata per essere utilizzata alla richiesta successiva.

Questo meccanismo, per quanto concettualmente semplice, è quello che permette concretamente a un chatbot di ricordare informazioni fornite in precedenza nel corso della stessa conversazione, come il nome della persona con cui sta interagendo o una preferenza già espressa in un messaggio precedente, un comportamento che contribuisce in modo determinante a rendere l'interazione percepita come naturale e coerente da parte di chi la utilizza.

4.3 Un ciclo di conversazione continuo

Per trasformare la logica appena descritta in un chatbot realmente utilizzabile in una conversazione continuativa, è necessario racchiuderla all'interno di un ciclo, una struttura di controllo del linguaggio Python già affrontata nei moduli introduttivi di questo percorso, che continui a richiedere un nuovo messaggio da parte della persona, aggiornare la cronologia, inviarla al modello, e mostrare la risposta ottenuta, ripetendo questo schema fino a quando la persona non decida di concludere la conversazione.

```
cronologia = [
    {"role": "system", "content":
istruzione_sistema}
]

while True:
    messaggio_utente = input("Tu: ")
```

```
    if messaggio_utente.lower() == "esci":
        break
    cronologia.append({"role": "user",
"content": messaggio_utente})
    risposta = client.chat.completions.create(
        model="gpt-4",
        messages=cronologia
    )
    testo_risposta =
risposta.choices[0].message.content
    print("Chatbot:", testo_risposta)
    cronologia.append({"role": "assistant",
"content": testo_risposta})
```

In pratica

- Conservare l'intera cronologia della conversazione in una lista di messaggi
- Aggiornare la cronologia dopo ogni messaggio ricevuto e ogni risposta ottenuta
- Racchiudere la logica di conversazione in un ciclo continuo
- Prevedere una condizione chiara per permettere di concludere la conversazione

5. Da script a riga di comando a interfaccia utilizzabile

5.1 I limiti di un chatbot a sola riga di comando

Il chatbot realizzato seguendo le indicazioni delle sezioni precedenti funziona correttamente, ma richiede, per essere utilizzato, l'esecuzione diretta dello script Python da una finestra a riga di comando, un'operazione che presuppone una minima familiarità tecnica non sempre presente in chi dovrebbe effettivamente utilizzare

il chatbot in un contesto lavorativo reale, come un collaboratore della propria attività o, ancora di più, un cliente esterno.

Per superare questo limite, come già accennato nel primo modulo di questo percorso dedicato alle librerie Python per l'IA generativa, è possibile ricorrere a librerie pensate specificamente per trasformare uno script Python in un'applicazione dotata di un'interfaccia grafica accessibile da un browser, senza richiedere competenze di sviluppo web tradizionale.

5.2 Costruire un'interfaccia grafica con Streamlit

Streamlit, già presentata nel primo modulo di questo percorso, rappresenta una delle soluzioni più immediate per trasformare il chatbot realizzato finora in un'applicazione web dotata di un campo di testo per inserire i propri messaggi e di un'area in cui visualizzare progressivamente lo scambio di messaggi avvenuto, riproducendo un'esperienza visivamente simile a quella degli strumenti di messaggistica più diffusi, familiare quindi a chiunque debba utilizzare il chatbot senza necessità di alcuna formazione specifica.

L'adattamento del codice già scritto per funzionare con Streamlit richiede alcune modifiche relativamente contenute rispetto alla versione a riga di comando: la cronologia della conversazione, ad esempio, deve essere conservata in una modalità compatibile con il funzionamento specifico di questa libreria, che ricarica lo script a ogni interazione della persona con l'interfaccia, un dettaglio tecnico gestito tramite un apposito meccanismo di conservazione dello stato messo a disposizione dalla libreria stessa, descritto nella relativa documentazione ufficiale.

5.3 Un esempio pratico di interfaccia semplificata

Di seguito è riportata una versione semplificata del codice del chatbot adattata per l'utilizzo con Streamlit, pensata esclusivamente a titolo dimostrativo per illustrare la struttura generale di questo tipo di adattamento, e non come codice completo pronto per un utilizzo in produzione, che richiederebbe ulteriori accorgimenti relativi, ad esempio, alla gestione degli errori già discussa nel modulo precedente.

```
import streamlit as st

st.title("Assistente virtuale")

if "cronologia" not in st.session_state:
    st.session_state.cronologia = [
        {"role": "system", "content":
istruzione_sistema}
    ]

messaggio = st.chat_input("Scrivi un messaggio")
if messaggio:
    st.session_state.cronologia.append(
        {"role": "user", "content": messaggio}
    )
    risposta = client.chat.completions.create(
        model="gpt-4",
        messages=st.session_state.cronologia
    )
    st.session_state.cronologia.append(
        {"role": "assistant", "content":
risposta.choices[0].message.content}
    )
```

In pratica

- Riconoscere i limiti di un chatbot utilizzabile solo da riga di comando

- Considerare Streamlit per offrire un'interfaccia grafica accessibile da browser
- Adattare la gestione della cronologia al funzionamento specifico della libreria scelta
- Trattare un primo esempio funzionante come base da perfezionare, non come prodotto finito

6. Migliorare, personalizzare e proseguire il proprio progetto

6.1 Affinare il comportamento tramite il prompt di sistema

Una volta ottenuto un chatbot funzionante nella sua struttura essenziale, gran parte del lavoro di perfezionamento successivo si concentra sull'affinamento dell'istruzione di sistema già descritta nella seconda sezione di questo modulo: osservando conversazioni reali o simulate, è possibile individuare comportamenti indesiderati, come risposte eccessivamente lunghe, un tono non coerente con la propria attività, o l'affrontare argomenti estranei allo scopo previsto, e correggerli intervenendo direttamente sul testo dell'istruzione fornita al modello, senza necessità di modificare la struttura complessiva del codice già sviluppato.

6.2 Gestire errori e casi limite

Come già sottolineato nel modulo dedicato all'utilizzo delle API, un'applicazione destinata a un utilizzo reale, anche da parte di un numero limitato di persone, deve prevedere una gestione adeguata dei possibili errori: nel contesto specifico di un chatbot, questo significa, ad esempio, mostrare un messaggio comprensibile qualora la richiesta al modello non vada a buon fine, invece di interrompere bruscamente la conversazione, e prevedere un comportamento

ragionevole qualora la persona invii un messaggio vuoto o privo di senso apparente.

Dedicare tempo a questo aspetto, spesso trascurato in un primo prototipo dimostrativo come quello descritto in questo modulo, contribuisce in modo significativo alla percezione di affidabilità del proprio chatbot da parte di chi lo utilizza concretamente, un fattore particolarmente rilevante qualora si preveda di renderlo disponibile a persone esterne alla propria attività, come i propri clienti.

6.3 Prossimi passi possibili per ampliare il progetto

Il chatbot realizzato seguendo le indicazioni di questo modulo rappresenta una base essenziale, ulteriormente ampliabile in diverse direzioni: collegare il chatbot a documenti specifici della propria attività, tramite l'approccio di retrieval augmented generation già introdotto nel primo modulo di questo percorso a proposito del framework LlamaIndex, permetterebbe al chatbot di rispondere basandosi su informazioni realmente proprie dell'attività invece che sulla sola conoscenza generale del modello; l'adozione di un framework più articolato come LangChain, anch'esso già presentato nel primo modulo, permetterebbe invece di gestire flussi di conversazione più complessi, superando i limiti della semplice struttura a lista di messaggi qui descritta.

Questi sviluppi più avanzati esulano dagli obiettivi di questo modulo introduttivo, pensato per fornire una prima esperienza concreta e completa di realizzazione di un chatbot funzionante, ma rappresentano naturali direzioni di approfondimento per chi, acquisita familiarità con i concetti di base qui illustrati, desideri sviluppare

ulteriormente le funzionalità del proprio progetto in base alle esigenze specifiche della propria attività.

In pratica

- Affinare progressivamente l'istruzione di sistema osservando conversazioni reali
- Prevedere una gestione adeguata degli errori prima di un utilizzo reale del chatbot
- Considerare la retrieval augmented generation per collegare il chatbot a documenti propri
- Valutare un framework come LangChain per gestire flussi di conversazione più complessi

7. Testare il proprio chatbot prima di renderlo disponibile

7.1 Perché il collaudo è un passaggio da non trascurare

Prima di rendere il proprio chatbot disponibile a colleghi, collaboratori o clienti, è opportuno dedicare un tempo adeguato a un collaudo sistematico del suo comportamento, un passaggio spesso sottovalutato da chi, dopo aver ottenuto un primo funzionamento soddisfacente su alcune prove informali, tende a considerare il progetto concluso: un modello linguistico, per sua stessa natura, può generare risposte diverse a fronte di richieste simili, ed è quindi opportuno verificarne il comportamento su un numero di casi sufficientemente rappresentativo delle situazioni che il chatbot dovrà effettivamente affrontare una volta in uso.

Un collaudo condotto con attenzione permette inoltre di individuare, prima che lo faccia una persona esterna alla propria attività, eventuali

comportamenti indesiderati del chatbot, come risposte inesatte su informazioni specifiche della propria attività che il modello non può conoscere autonomamente, o un tono percepito come poco adeguato al contesto in cui il chatbot verrà utilizzato.

7.2 Costruire un piccolo insieme di conversazioni di prova

Un metodo pratico ed efficace per condurre questo collaudo consiste nel predisporre, prima ancora di considerare concluso lo sviluppo, un piccolo elenco scritto di messaggi rappresentativi delle richieste che le persone rivolgeranno presumibilmente al chatbot una volta operativo, comprendendo sia le richieste più prevedibili e frequenti, sia alcune richieste ai margini dello scopo previsto, utili a verificare che il chatbot mantenga correttamente i vincoli di comportamento definiti nell'istruzione di sistema anche di fronte a sollecitazioni meno consuete.

Ripetere periodicamente questo stesso insieme di conversazioni di prova, specialmente dopo ogni modifica apportata all'istruzione di sistema o al codice del chatbot, permette di verificare che le modifiche introdotte abbiano effettivamente migliorato il comportamento complessivo, senza al contempo introdurre nuovi problemi in aspetti che in precedenza funzionavano correttamente, un rischio non trascurabile quando si interviene ripetutamente su uno stesso progetto nel corso del tempo.

7.3 Raccogliere un riscontro da persone reali

Oltre al collaudo condotto autonomamente da chi sviluppa il chatbot, un passaggio ulteriore particolarmente utile consiste nel far provare una prima versione del chatbot a un piccolo gruppo di persone reali, come alcuni colleghi della propria attività, prima di renderlo

disponibile su più ampia scala: chi non ha partecipato allo sviluppo del chatbot tende infatti a porre domande e a utilizzare un linguaggio meno prevedibile rispetto a chi lo ha progettato, mettendo in luce comportamenti inattesi che difficilmente emergerebbero da un collaudo condotto in autonomia.

Questo riscontro esterno, per quanto raccolto in modo informale, rappresenta un'occasione preziosa per affinare ulteriormente l'istruzione di sistema e, più in generale, per valutare se il chatbot realizzato risponda effettivamente allo scopo individuato nella fase di progettazione iniziale descritta nella seconda sezione di questo modulo, prima di investire ulteriore tempo nello sviluppo delle funzionalità più avanzate accennate nella sezione precedente.

In pratica

- Dedicare un tempo adeguato al collaudo prima di rendere il chatbot disponibile
- Predisporre un piccolo insieme scritto di conversazioni di prova rappresentative
- Ripetere il collaudo dopo ogni modifica rilevante al codice o alle istruzioni
- Raccogliere un riscontro da persone reali esterne allo sviluppo del progetto

8. Alcuni errori comuni da evitare nei primi progetti

8.1 Istruzioni di sistema troppo generiche o troppo rigide

Tra gli errori più frequenti riscontrabili nei primi progetti di chatbot vi è la scrittura di un'istruzione di sistema eccessivamente generica,

come una semplice indicazione a rispondere in modo cortese, priva di riferimenti concreti allo scopo specifico del chatbot già individuato nella fase di progettazione: un'istruzione di questo tipo tende a produrre un comportamento poco distintivo, non sostanzialmente diverso da quello di un modello linguistico generico privo di alcuna configurazione specifica.

L'errore opposto, altrettanto frequente, consiste in un'istruzione eccessivamente rigida e dettagliata, che tenta di prevedere in anticipo ogni possibile situazione tramite un elenco molto lungo di regole specifiche: questo approccio rischia non soltanto di appesantire inutilmente ogni richiesta inviata al modello, con un conseguente aumento dei costi già discusso nel modulo precedente, ma anche di produrre un comportamento innaturale, qualora le regole elencate risultino tra loro poco coerenti o difficili da conciliare in una singola risposta.

8.2 Trascurare la cronologia della conversazione

Un secondo errore comune, specialmente per chi si avvicina per la prima volta a questo tipo di sviluppo, consiste nel sottovalutare l'importanza di una corretta gestione della cronologia della conversazione già descritta nella quarta sezione di questo modulo: un errore nella costruzione di questa lista di messaggi, ad esempio dimenticando di aggiungervi la risposta generata dal modello prima della richiesta successiva, produce un chatbot apparentemente funzionante nelle singole interazioni isolate, ma privo della coerenza conversazionale che ne rappresenta uno dei principali punti di forza rispetto ai chatbot tradizionali basati su regole predefinite.

Verificare esplicitamente, nel corso del collaudo già descritto nella sezione precedente, che il chatbot faccia effettivamente riferimento a informazioni fornite in messaggi precedenti della stessa conversazione rappresenta quindi un controllo semplice ma particolarmente efficace per individuare tempestivamente questo tipo di errore, prima che diventi evidente a chi utilizzerà il chatbot una volta reso disponibile.

In pratica

- Evitare istruzioni di sistema troppo generiche, prive di riferimento allo scopo specifico
- Evitare istruzioni eccessivamente lunghe e dettagliate, difficili da conciliare tra loro
- Verificare con attenzione la corretta costruzione della cronologia dei messaggi
- Controllare, in fase di collaudo, che il chatbot faccia riferimento a messaggi precedenti

In sintesi

La realizzazione di un primo chatbot funzionante in Python richiede la messa in pratica combinata dei concetti già illustrati nei moduli precedenti di questo percorso: la scelta e l'installazione delle librerie necessarie, l'utilizzo corretto dell'API del modello linguistico prescelto, la costruzione di un'istruzione di sistema efficace, e la gestione di una cronologia di messaggi che permetta al chatbot di sostenere una conversazione coerente nel tempo, superando il limite dell'assenza di memoria autonoma propria dei modelli linguistici.

Da questa struttura essenziale, illustrata passo dopo passo in questo modulo attraverso esempi di codice via via più completi, è possibile

procedere verso un'interfaccia grafica accessibile a chiunque tramite librerie come Streamlit, e successivamente verso funzionalità più avanzate, come il collegamento a documenti specifici della propria attività o l'adozione di framework più articolati, già introdotti nei moduli precedenti di questo percorso e approfonditi ulteriormente nei moduli successivi.